A World Model as a Judge: Early, Auditable Verdicts on Partial Robot-Manipulation Rollouts

Abdelhamid Bakhta
StarkWare
abdel@starkware.co

July 13, 2026

Companion paper to `leworldmodel-judge` v0.2.0*

Abstract

We present **LeWorldModel Judge**, a locked benchmark and an auditable judging surface for scoring partial robot-manipulation rollouts before their episodes end. Sparse terminal reward answers no useful question about a trajectory prefix: whether the rollout is still on track, already doomed, or physically implausible only becomes visible in hindsight, which makes early triage, data curation, and process-reward experiments needlessly blind. We lock a narrow evaluation slice (three Meta-World tasks, four constructed policy families, prefix cutoffs at 0.25, 0.50, and 0.75 of the horizon), define heuristic but fully inspectable prefix labels, and score every prefix with a cutoff-time composite judge over named evidence signals, alongside a replay-time latent proxy whose causal limitation we state exactly. A mandatory calibration protocol reports the provenance of every decision threshold and only grants held-out status when the calibration and evaluation policy families are disjoint. On a fresh 90-prefix held-out slice collected after the label rules were frozen, the composite judge reaches a 0.949 failure hit rate at a 0.137 false positive rate and 0.978 pairwise ranking accuracy, against 0.50 and 0.554 for sparse-reward and progress baselines, and the threshold calibrated on disjoint families transfers across disjoint captures to within 0.0004 (0.2977 versus 0.2980). Every reported number decomposes to named sub-scores in checked-in files, carries a `judge_mode` provenance tag, and regenerates from a pinned command line.

1 Introduction

A sparse success signal is silent until it is too late. In manipulation benchmarks the reward that matters is terminal: the object ends in place or it does not, and until that moment the learner, the operator, and any data-curation pipeline sitting between them are equally uninformed. Yet most of the questions practitioners actually ask concern the middle of a rollout. Is this trajectory still likely to succeed? Has it already become unrecoverable? Does the state evolution look physically wrong? How much should we trust any of those calls?

This paper describes a deliberately narrow system that answers those questions on a locked benchmark slice and, just as deliberately, refuses to answer anything wider. The project is anchored in the world-model research program associated with joint-embedding predictive architectures (LeCun, 2022;

*Code, benchmark artifacts, and reproduction commands: https://github.com/AbdelStark/leworldmodel-judge. Published cloud runs with full provenance: https://huggingface.co/datasets/abdelstark/leworldmodel-judge-runs.

Assran et al., 2023; Bardes et al., 2024): a world model that predicts how an episode should continue supplies a natural surprise signal for judging how an episode is actually going. The shipped implementation, however, is not a trained world model, and we do not present it as one. It is a hand-weighted composite over decomposed prefix evidence plus a linear observation-space proxy, and the distinction between the conceptual anchor and the shipped mechanism is maintained everywhere, including in the per-row provenance tags of the released data.

The contribution is best read as a benchmark-and-protocol result rather than a modeling result:

- **A locked benchmark for prefix-level trajectory verification.** Three Meta-World tasks (Yu et al.), four constructed policy families (expert, weak, doomed, misleading), prefix cutoffs at fractions 0.25, 0.50, and 0.75 of the episode horizon, heuristic task-aware failure and recoverability labels, and a fixed baseline set. The slice is small by design: every claim in this paper traces to files small enough to read.
- **An auditable judging surface.** Every verdict is serialized with the seven named evidence fields of the record contract, joined by key to the prefix record that holds the remaining raw inputs, every row carries a `judge_mode` tag naming the implementation that produced it, and a null judge with the same record shape sanity-checks the harness. The composite judge is cutoff-time: it reads nothing past the cutoff. A hybrid variant that consumes a latent proxy is replay-time by construction, and we treat that causal boundary as part of the result rather than a footnote.
- **A calibration protocol with mandatory provenance.** Every reported threshold states whether it was fixed in advance, tuned in-slice, or calibrated on a held-out family split, and held-out status is only granted when calibration and evaluation families are disjoint. Reviewer-facing summaries name the cohorts and their label counts.
- **Evidence that the protocol does useful work.** On the initial 18-prefix held-out slice the judge separates perfectly, which proves wiring rather than generalization. On a fresh capture collected after the label rules were frozen, with five times the evaluation cohort, perfect separation degrades honestly (hit rate 0.949, false positive rate 0.137) while the ranking gap over sparse-reward and progress baselines persists (0.978 versus 0.50 and 0.554), and the calibrated operating point transfers across disjoint captures to within 0.0004.

We also report the negative and confounded results with the same prominence. No episode in either real capture reaches terminal success within the 75-step cap, so on real data every detection metric rests on the heuristic prefix labels rather than environment truth. The labeler and the judge draw on the same evidence family, and a single shared feature (distance regret) achieves 0.824 pairwise accuracy on the fresh held-out slice against the judge's 0.978, which bounds how much of the ranking signal is attributable to the composite rather than to its strongest input. Section 6 discusses both issues.

2 Related work

World models and predictive representations. Learning a model of environment dynamics and using its predictions as a training or planning substrate is a long-standing program (Ha and Schmidhuber, 2018; Hafner et al., 2020, 2023; Hansen et al., 2022). Joint-embedding predictive architectures move prediction into representation space: predict the embedding of a missing view rather than its pixels, whether masked regions of the same image (Assran et al., 2023), masked spatio-temporal regions of a video (Bardes et al., 2024), or, in the proposed full architecture, future world states (LeCun, 2022). Our work borrows the

judging intuition from this line (a predictor's disagreement with reality is evidence about the trajectory) without yet training a predictor; the shipped latent proxy is a linear extrapolation over mean-pooled observation windows, and we label it as such.

Surprise as a signal. Prediction error has served as an intrinsic reward for exploration (Pathak et al., 2017; Burda et al., 2018) and as an ensemble-disagreement objective for model-based exploration (Sekar et al., 2020). We invert the sign of the argument: rather than seeking surprising states, we ask whether a partial rollout is consistent with a predicted successful continuation, and treat inconsistency as evidence of failure.

Verifiers and process reward models. Outcome-supervised verifiers rerank candidate solutions by predicted correctness (Cobbe et al., 2021), and process supervision scores intermediate steps rather than final answers (Uesato et al., 2022; Lightman et al., 2024). Preference-based reward models occupy the same trust-critical position in RLHF pipelines (Christiano et al., 2017). Our judge is the embodied analogue of a process reward signal, with one additional design constraint these lines rarely impose: every score must decompose into inspectable evidence so that a skeptical reviewer can challenge an individual verdict without rerunning anything.

Failure detection in manipulation. Dedicated failure detectors for manipulation include multimodal sensor-fusion classifiers (inc), vision-language success detectors (Du et al., 2023), and LLM-based post-hoc failure explanation (liu). These systems mostly classify completed or nearly completed executions; our benchmark targets the earlier question of what can be said at fixed fractional cutoffs of a still-running episode, and evaluates ranking as well as detection.

Evaluating without executing. Off-policy evaluation estimates policy value from logged data (Thomas and Brunskill, 2016; Levine et al., 2020). Prefix judging is a complementary, more granular problem: not the value of a policy, but the fate of one partially observed trajectory. We use standard threshold-free ranking statistics precisely because operating points chosen on the evaluation slice are the classic failure mode of such comparisons.

3 The benchmark

3.1 Environment slice and policy families

The benchmark locks three Meta-World tasks (Yu et al.) on MuJoCo physics (Todorov et al.) behind the Gymnasium interface (Towers et al., 2024): `reach-v3`, `push-v3`, and `pick-place-v3`. Rollouts come from scripted expert policies degraded per *policy family*, so that failure modes exist by construction rather than by accident:

- **expert**: the scripted policy, unmodified;
- **weak**: actions scaled to 0.55 of expert magnitude with shared-scalar noise on the translation dimensions;
- **doomed**: expert actions negated and rescaled after 40% of the horizon;
- **misleading**: expert behavior that switches to zero actions after 30% of the horizon, so early evidence looks healthy and late evidence is dead.

A fifth family (**random**) exists in the collector as a negative control and is not part of the reported slices. Episodes are captured at up to 75 steps; a capture is 5 episodes per (task, family) pair in the fresh run reported here (60 episodes, 4,500 step records) and 1 episode per pair in the initial capture (12 episodes). Meta-World physics is not byte-reproducible across simulator builds, so each capture's `rollouts.jsonl` is checked in as the input of record and everything downstream regenerates deterministically from it.

3.2 Prefixes, labels, and what the labels are not

Each episode is sliced at fractional cutoffs $f \in \{0.25, 0.50, 0.75\}$ of its realized horizon. A prefix record carries the evidence signals available up to the cutoff, derived from Meta-World info fields: distance progress and its decomposition (start, best, and last distance to target), an in-place score, near-object and grasp signal peaks, a success signal peak, reward density (mean non-negative unscaled reward), a stall score, and a progress proxy. Missing signals remain `null`; absence is never coerced to zero.

Each prefix receives two labels from a task-aware heuristic labeler: `prefix_failure_label` (boolean: effectively doomed at this cutoff) and `prefix_recoverability_label` (recoverable, at risk, or doomed). The rules are deliberately conservative: episodes that end in success are never failure-labeled, a prefix with no observed state signal is at most at risk, and no prefix is labeled doomed before cutoff 0.5. Task-specific gates handle the ways each task actually dies; the `push-v3` gates were hardened against noisy contact signals masking dead trajectories.

Two properties of these labels matter for interpreting every number that follows. First, they are heuristics over the same evidence family the judge reads, not independent ground truth; only `final_success_label` is environment truth, and it only vetoes failure labels on successful episodes. Second, in both real captures reported here, no episode reaches terminal success within the 75-step cap, so the veto never fires and the real-data metrics rest entirely on the heuristic labels. We measure the practical consequence of this circularity in Section 5.3 rather than asking the reader to take it on faith.

3.3 Baselines, metrics, and calibration protocol

Every prefix is scored by three baselines on the same records as the judge: cumulative sparse reward over the prefix (absence of reward read as predicted failure), a progress proxy against a fixed threshold of 0.2, and terminal success, which is excluded from the comparison as an oracle upper bound because it is not available at the cutoff.

Reported metrics per slice: failure hit rate (fraction of failure-labeled prefixes flagged), false positive rate, pairwise ranking accuracy (the probability that a uniformly drawn failure-labeled prefix outscores a uniformly drawn unlabeled one, ties counted half, which equals AUROC), and average precision. Metrics pool the three cutoffs; per-cutoff detection lead time is future work and we do not claim it.

The failure threshold applied to the judge must state its provenance in every summary file: *fixed* (chosen before looking at the slice), *in-slice* (tuned by balanced accuracy on the reported slice; a debugging operating point, never a deployment claim), or *held-out* (`held_out_family_split`: tuned on a calibration cohort of policy families disjoint from the evaluation families). The evaluator refuses to emit held-out provenance when the family sets overlap. Baselines are deliberately not calibrated; the false positive row therefore compares a calibrated judge operating point against uncalibrated baseline defaults, and the threshold-free ranking rows carry the calibration-fair comparison.

4 The judge

4.1 Output contract

For a prefix with evidence vector E, a judge emits four scores in $[0, 1]$: `on_track`, `failure`, `implausibility`, and `uncertainty`, together with seven named evidence fields (progress, distance progress, in-place, near-object, grasp, reward, stall) and a `judge_mode` tag naming the implementation. The remaining raw inputs, including the target distances behind the distance-regret term and the success signal peak, live in the prefix record checked in beside the judge rows and joined by the same key; auditing a verdict end to end reads the pair, not the judge row alone. A null judge (all scores zero, uncertainty one) shares the record shape so the harness can be checked against a signal-free scorer. The judge never reads the labels; a regression test pins that flipping every label leaves every score unchanged.

4.2 Composite cutoff-time judge

The headline judge (`composite_prefix_judge`) derives intermediate features from the evidence: target proximity, distance regret (how far the object has drifted back from its best distance), engagement (near-object or grasp), transport (in-place, success, or distance progress), early patience (engaged early prefixes are forgiven), stalling without transport, and engagement without transport. The failure score is a hand-weighted linear combination of these features clipped to $[0, 1]$; the weights are frozen constants in the released code, and we make no claim that they are optimal. What we do claim is the causal property: the composite judge reads only prefix evidence, so its verdict is computable on a live rollout at the moment of the cutoff.

4.3 Latent proxy and the replay-time boundary

The hybrid judge (`hybrid_prefix_latent_judge`) adds a surprise term derived from an intentionally simple latent cache. For an episode with observations $o_{1:T}$ and a prefix of length k:

$$z_{\text{ctx}} = \tfrac{1}{k} \textstyle\sum_{t \le k} o_t, \qquad \bar{\Delta} = \tfrac{1}{k-1} \textstyle\sum_{t<k} (o_{t+1} - o_t),$$
$$\hat{z}_{\text{fut}} = z_{\text{ctx}} + (T-k)\,\bar{\Delta}, \qquad z_{\text{fut}} = \tfrac{1}{T-k} \textstyle\sum_{t>k} o_t,$$

and the mismatch score is the mean absolute gap between $\hat{z}_{\text{fut}}$ and z_{fut}, clipped to $[0, 1]$. High mismatch raises the failure, implausibility, and uncertainty scores and lowers the on-track score relative to the composite row.

This construction is an observation-space placeholder for a learned encoder, and it has a causal asymmetry that we treat as a first-class result: z_{fut} reads observations after the cutoff, so the hybrid score exists only in replay. It is a hindsight consistency check for completed episodes, useful for triage, and it is not an early-verdict signal. The same standard excludes terminal success from the baseline set. On the initial held-out capture the distinction costs nothing: the composite judge reproduces the hybrid headline metrics exactly, with the latent term moving individual failure scores by at most +0.036 (mean +0.011). Any early-verdict claim in this paper therefore rests on the composite judge alone.

5 Results

All numbers below are read from checked-in summary files in the repository; each carries its `judge_mode` and threshold provenance, and the captions name the source file. We report three slices: the canonical

Table 1: Composite judge versus baselines under the held-out family split (calibrated on weak and doomed families, evaluated on expert and misleading). Left: initial capture (`hard-family-real-held-out-2026-04-28`, evaluation cohort n=18, threshold 0.298006). Right: fresh capture collected with frozen label rules (`hard-family-real-fresh-capture-2026-07-13`, n=90, threshold 0.297680). All judge rows are `composite_prefix_judge`; pairwise accuracy equals AUROC.

	Initial capture (n=18)			Fresh capture (n=90)		
Metric	Judge	Sparse	Progress	Judge	Sparse	Progress
Failure hit rate ↑	**1.000**	1.000	1.000	0.949	**1.000**	**1.000**
False positive rate ↓	**0.100**	1.000	1.000	**0.137**	1.000	1.000
Pairwise accuracy ↑	**1.000**	0.500	0.556	**0.978**	0.500	0.554
Average precision ↑	**1.000**	0.444	0.485	**0.971**	0.433	0.463

synthetic benchmark, the initial real held-out capture, and a fresh real capture collected after the label rules were frozen.

5.1 Held-out family split on real captures

Table 1 contains the central comparison. Three observations, in decreasing order of importance.

The ranking gap survives a fresh capture at five times the cohort. Both baselines flag every prefix on these slices (their false positive rate is 1.0), so raw detection is uninformative; the judge's value is concentrated in false positives and ranking quality. On the fresh capture the composite judge holds 0.978 pairwise accuracy against 0.500 for sparse reward and 0.554 for the progress proxy, and 0.971 average precision against 0.433 and 0.463 (Figure 1).

The calibrated operating point transfers across captures. Calibrating on the fresh capture's weak and doomed cohort (90 prefixes, 32 failure-labeled) yields threshold 0.297680; the same procedure on the initial capture's 18-prefix cohort had yielded 0.298006. Disjoint episodes, seeds, and cohort sizes recover the same operating point to within 4×10^{-4} (Figure 3). We did not expect agreement this tight and make no claim it will persist to other tasks, but it is evidence that the threshold reflects structure in the score distribution rather than noise in one capture.

Perfect separation does not survive scale, and the errors have structure. The initial capture's 1.000 rows are perfect separation on a tiny slice; the fresh capture relaxes them to a 0.949 hit rate and 0.137 false positive rate. The two missed failures both live in `push-v3` misleading prefixes at cutoff 0.50 (per-task hit rate 0.833 there), and six of the seven false positives live in `pick-place-v3` (per-task false positive rate 0.261): four are late unlabeled prefixes that drift above the threshold at cutoff 0.75, and three are `pick-place-v3` expert prefixes already marginally above it at cutoff 0.50 (Figure 2). This is the degradation the fresh capture existed to expose; a benchmark that stayed perfect at five times the cohort would be describing its labeler, not its judge.

The hybrid replay-time judge on the same fresh split scores a 0.923 hit rate, 0.137 false positive rate, 0.976 pairwise accuracy, and 0.970 average precision at threshold 0.313437 (`hybrid_prefix_latent_judge`, `summary.json`). Consistent with Section 4.3, the latent term adds nothing over the composite here; its value, if any, is in replay triage.

5.2 Synthetic benchmark

The synthetic slice (24 episodes, 72 prefixes, in-slice threshold 0.360053, failure-label coverage 4 of 72) is the controlled proof surface: the generator constructs the failure modes, so it isolates judge behavior

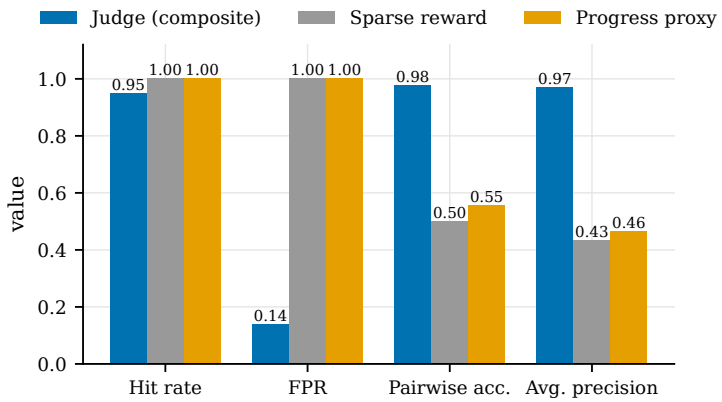

Figure 1: Composite judge versus sparse-reward and progress baselines on the fresh capture's held-out evaluation slice (n=90; source `summary-composite.json`). Both baselines flag every prefix, so their hit rate and false positive rate are both 1.0; the judge trades five points of hit rate for an 86-point false positive improvement, and the threshold-free ranking metrics carry the calibration-fair comparison.

from capture noise. The composite judge scores a 1.0 hit rate at 0.029 false positive rate with 0.985 pairwise accuracy; sparse reward again ranks at chance (0.500) and the progress baseline ranks these families badly (0.147). The threshold is in-slice and we report it as a debugging operating point, not an operating claim. A held-out family split on synthetic data is degenerate by construction: the expert and misleading evaluation families contain no failure-labeled prefixes there (synthetic expert and weak episodes succeed by design, and synthetic misleading episodes stay recoverable under the conservative labeler), so held-out evaluation metrics would honestly return `null`, and the held-out story belongs to the real captures.

5.3 How circular are the labels?

The labeler and the judge consume the same evidence family, so judge-versus-label agreement is partly built in. We quantify the overlap instead of hand-waving it. Distance regret, the single strongest shared feature, ranked with the same tie-handled pairwise statistic, scores 0.985 on the synthetic slice, matching the judge's headline there; on real captures it scores 0.825 (initial, n=18) and 0.824 (fresh, n=90) against the judge's 1.000 and 0.978. Three readings follow. First, on synthetic data the benchmark cannot distinguish the composite from its strongest input, and synthetic separation should be discounted accordingly. Second, on real captures the composite adds roughly fifteen points of pairwise accuracy over its strongest single feature, so the aggregation is doing measurable work beyond one input. Third, all of these comparisons remain within the shared evidence family; labels independent of the judge's inputs (human annotation, or environment-defined failure states) are the only way to close the question, and they are future work, not shipped.

5.4 Reproducibility and cloud runs

Every artifact directory carries the capture of record (`rollouts.jsonl`), every derived table, both judge outputs, both summaries, and rendered reports; deterministic stages regenerate byte-identically on the pinned interpreter (CPython 3.11). The fresh capture reported here was executed end to end on Hugging Face Jobs CPU hardware: the launcher refuses dirty or unpushed working trees, pins the remote instal-

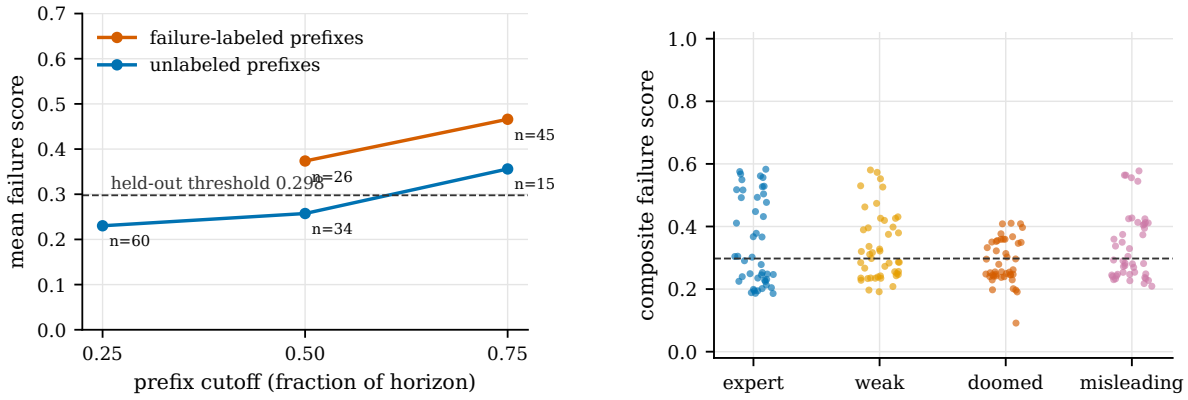

Figure 2: Left: mean composite failure score by cutoff on the fresh capture, split by the heuristic prefix failure label. The labeler emits no failure label before cutoff 0.5 by rule. Failure-labeled prefixes sit above the held-out threshold from the first cutoff at which they exist; the unlabeled mean crosses it at cutoff 0.75, where four of the seven false positives live (the other three sit marginally above threshold at cutoff 0.50). Right: every composite failure score in the fresh capture by policy family (180 prefixes; jitter is presentational). The families overlap substantially; the judge is not a family classifier, and the held-out split is doing real work.

lation to the exact commit under review, and gates the published run folder against a 22-check contract (checksums of every file, row-count joins, `judge_mode` values, and calibration provenance) before it can be cited. The full pipeline for the 60-episode capture took 24.7 seconds of stage wall time; the run's `provenance.json` records the argv, timing, and SHA-256 of every stage and file, and the published copy is public.[1] Total compute cost for the full run matrix was under one cent, which we mention not as an efficiency claim but as evidence that nothing about the benchmark requires trust in unreproducible infrastructure.

6 Limitations

The slices are small and narrow. Ninety held-out evaluation prefixes from sixty episodes across three tasks in one simulator is enough to falsify perfect separation and measure a ranking gap; it is not enough to support claims about manipulation in general, other simulators, or real robots, and we make none.

The labels are heuristics over the judge's own evidence family. Section 5.3 bounds the overlap but cannot eliminate it. Worse, in both real captures no episode reaches terminal success within the 75-step cap, so the environment-truth veto never fires on real data and every real-data metric rests on the heuristic labels. The honest reading of Table 1 is therefore: the judge ranks prefixes by imminent failure as a conservative task-aware labeler defines it, substantially better than sparse reward does, under thresholds that transfer across captures. Whether that ordering matches human judgment of doom is unmeasured.

The judge is a hand-weighted heuristic and the latent proxy is not a world model. No learned encoder, no trained dynamics, no representation learning of any kind ships in these results. The project's name states its research program, not its implementation status, and the per-row `judge_mode` tags exist precisely so the two cannot be confused. The hybrid variant is additionally replay-time only.

Detection is not the judge's win on these slices; nor is lead time measured. Both baselines reach hit rate 1.0 by flagging everything, so the win is false positives and ranking. Metrics pool the three cutoffs;

[1] Run `metaworld-benchmark-20260713-080743-g666670e` under https://huggingface.co/datasets/abdelstark/leworldmodel-judge-runs.

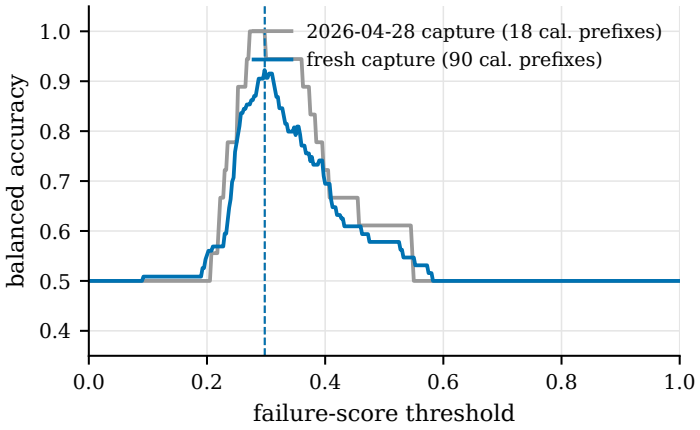

Figure 3: Balanced accuracy over failure-score thresholds on the calibration cohorts (weak and doomed families) of the two real captures, computed from the checked-in prefix and judge files. Dashed lines mark each capture's selected threshold: 0.298006 (initial, grey) and 0.297680 (fresh, blue). Disjoint captures recover the same operating point to within 4×10^{-4}.

"early" is structural (labels and scores exist at fixed early cutoffs) rather than a measured detection lead time, and per-cutoff lead-time analysis is future work.

Calibration is asymmetric. Only the judge threshold is calibrated; the progress baseline keeps a fixed default and sparse reward is binary. The threshold-free rows are the fair comparison, and they are the rows the conclusions rest on.

7 Conclusion

We set out to test a narrow proposition: that a world-model-motivated, fully auditable surprise signal can judge partial manipulation rollouts earlier and more usefully than sparse reward, under an evaluation protocol that states the provenance of every threshold and measures the circularity it cannot remove. On the locked slice the proposition survives contact with a fresh capture: the ranking gap over sparse reward persists at five times the evaluation cohort, the calibrated threshold transfers across disjoint captures, and the places where the judge degrades are identifiable in the released files down to individual prefixes. The protocol contributions (family-held-out threshold calibration with mandatory provenance, per-row judge-mode tags, the cutoff-time versus replay-time boundary, and a verify-gated artifact contract) are independent of the current heuristic judge and are, we think, the durable part of this work. The obvious next step is the one the project's name promises: replace the mean-pooled observation proxy with a learned encoder and a continuation predictor that is computable at the cutoff, then hold it to the same benchmark, the same provenance rules, and the same willingness to publish degradation.

Acknowledgments

The cloud execution path runs on Hugging Face Jobs, and parts of the operational tooling were exercised by Hugging Face's ml-intern agent under scoped prompts, with transcripts published alongside the runs. All analysis, claims, and errors are the author's.

References

FINO-Net.

REFLECT.

Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture, 2023. URL https://arxiv.org/abs/2301.08243.

Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video, 2024. URL https://arxiv.org/abs/2404.08471.

Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation, 2018. URL https://arxiv.org/abs/1810.12894.

Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pages 4299–4307, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/d5e2c0adad503c91f91df240d0cd4e49-Abstract.html.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL https://arxiv.org/abs/2110.14168.

Yuqing Du, Ksenia Konyushkova, Misha Denil, Akhil Raju, Jessica Landon, Felix Hill, Nando de Freitas, and Serkan Cabi. Vision-language models as success detectors, 2023. URL https://arxiv.org/abs/2303.07280. Published at CoLLAs 2023.

David Ha and Jürgen Schmidhuber. World models, 2018. URL https://arxiv.org/abs/1803.10122.

Danijar Hafner, Timothy P. Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020. URL https://openreview.net/forum?id=S1lOTC4tDS.

Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models, 2023. URL https://arxiv.org/abs/2301.04104. Journal version: Mastering diverse control tasks through world models, Nature 640:647–653 (2025), doi:10.1038/s41586-025-08744-2.

Nicklas A Hansen, Hao Su, and Xiaolong Wang. Temporal difference learning for model predictive control. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato, editors, *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 8387–8406. PMLR, 17–23 Jul 2022. URL https://proceedings.mlr.press/v162/hansen22a.html.

Yann LeCun. A path towards autonomous machine intelligence. OpenReview, 2022. URL https://openreview.net/forum?id=BZ5a1r-kVsf.

Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems, 2020. URL https://arxiv.org/abs/2005.01643.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL https://openreview.net/forum?id=v8L0pN6EOi.

Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction, 2017. URL https://arxiv.org/abs/1705.05363.

Ramanan Sekar, Oleh Rybkin, Kostas Daniilidis, Pieter Abbeel, Danijar Hafner, and Deepak Pathak. Planning to explore via self-supervised world models, 2020. URL https://arxiv.org/abs/2005.05960.

Philip S. Thomas and Emma Brunskill. Data-efficient off-policy policy evaluation for reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML 2016)*, volume 48 of *JMLR Workshop and Conference Proceedings*, pages 2139–2148. JMLR.org, 2016. URL http://proceedings.mlr.press/v48/thomasa16.html.

Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo.

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments, 2024. URL https://arxiv.org/abs/2407.17032.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback, 2022. URL https://arxiv.org/abs/2211.14275.

Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-World.

A Reproduction commands

The pipeline is a single CLI (`lewm-judge`, or `python -m leworldmodel_judge`) with one subcommand per stage. The fresh capture reported in Section 5.1 regenerates its derived files from the checked-in capture of record:

```
RUN=artifacts/hard-family-real-fresh-capture-2026-07-13
lewm-judge prefixes  --input $RUN/rollouts.jsonl --output $RUN/prefixes.jsonl
lewm-judge latents   --rollouts $RUN/rollouts.jsonl \
                     --prefixes $RUN/prefixes.jsonl \
                     --output $RUN/latent-cache.jsonl
lewm-judge baselines --input $RUN/prefixes.jsonl --output $RUN/baselines.jsonl
lewm-judge judge     --input $RUN/prefixes.jsonl --mode heuristic_surprise \
                     --output $RUN/judge-composite.jsonl
lewm-judge judge     --input $RUN/prefixes.jsonl --mode hybrid_surprise \
                     --latent-cache $RUN/latent-cache.jsonl \
```

```
                    --output $RUN/judge-hybrid.jsonl
lewm-judge evaluate --prefixes $RUN/prefixes.jsonl \
                    --baselines $RUN/baselines.jsonl \
                    --judge $RUN/judge-composite.jsonl \
                    --calibration-families weak,doomed \
                    --evaluation-families expert,misleading \
                    --output $RUN/summary-composite.json
```

The capture itself was collected on Hugging Face Jobs with the repository launcher (`uv run jobs/launch.py launch --preset metaworld-benchmark`), which is equivalent to `lewm-judge collect --source metaworld --task all --episodes 5 --max-steps 75 --seed 1013 --policy-family expert,weak,doom` followed by the chain above. Meta-World capture is not byte-reproducible across simulator builds; rerunning reproduces the protocol, not the bytes, which is why the capture is checked in.

Every figure in this paper regenerates from the checked-in artifacts with `uv run docs/paper/figures/make_figure` and the paper builds with `make -C docs/paper`.